Rapport de Conception

Roki

Moins de mails, plus de Roki

Date	26/09/2025	Organisation
Membre	Abdou Imane	Responsable Compte rendu
	Hoarau Thomas	DEV Front-end
	Houmadi Soihiri	DEV Back-end (Chefs d'équipe)

Table des matières

1.Cahier des charges..... 3
1.1 Présentation..... 3
1.2 Objectifs clairement définis..... 3
1.3 Fonctionnalités (principales et opérationnelles)..... 3
Fonction principales :..... 3
Fonction Opérationnelle :..... 3
1.4 Contraintes identifiées..... 4
1.5 Schémas de L'architecture du Projet..... 4
1.6 Diagrammes de Classes Backend..... 5
1.7 Cas d'utilisation..... 6
1.8 Diagrammes de séquence..... 7
1.9 Choix technologiques..... 11
1.10 Justification des langages..... 11
1.11 Frameworks..... 12
1.12 Persistance des données (JSON)..... 12
1. l'organisation du travail..... 12
1.1 répartition des tâches..... 12
1.2 Tableau de suivi du projet..... 14
2. l'état d'avancement technique du serveur et du client..... 16
2.1 architecture mise en place..... 16
2.1.1 Etat d'avancement du Serveur..... 16
2.1.2 Etat d'avancement du Client..... 16
2.1.3 Etat d'avancement Global..... 16
2.2 fonctionnalités déjà implémentés..... 18
2.3 captures d'écran..... 19
2.3.1 Page d'accueil..... 19
2.3.2 Page de connexion..... 20
2.3.3 Page d'inscription..... 20
2.3.4 Tableau de bord..... 21
2.4 aperçu du code..... 22
2.4.1 Modèle de données utilisateur (User Model)..... 22
2.4.2 Point d'entrée du serveur backend (server.ts)..... 23
2.4.4 Connexion à la base de données (db.ts)..... 24
2.4.5 Contrôleur de présence du token..... 25
2.4.6 Exemple de route protégée..... 26
2.4.7 Mise en place du CORS..... 26

2.4.8 Vérification du token utilisateur.....27
2.4.9 Authentification de l'utilisateur.....28
2.4.10 Dashboard utilisateur (Dashboard.tsx).....30
2.4.11 Page de connexion utilisateur (Login.tsx).....33

1.Cahier des charges

1.1 Présentation

Roki est une application de communication en ligne imaginée par Houmadi Soihiri.
Dans le cadre du projet SAE 302, notre équipe de développement, composée de Thomas Hoarau et Abdou Imane, a pour mission de concevoir, développer et améliorer cette plateforme afin d'en faire une solution moderne et accessible pour la communication en ligne.

L'objectif est de proposer un outil intuitif et adapté à tous, avec une attention particulière portée aux besoins des étudiants.

1.2 Objectifs clairement définis

- Créer une application de communication simple et efficace.
- Offrir une expérience utilisateur fluide et accessible à tous.
- Faciliter la communication entre étudiants (travaux de groupe, échanges rapides).
- Poser des bases solides permettant l'ajout progressif de nouvelles fonctionnalités.

1.3 Fonctionnalités (principales et opérationnelles)

Fonction principales :

- **Création de compte utilisateur** : chaque utilisateur dispose d'un espace personnel sécurisé lui permettant d'accéder à ses conversations et de gérer son profil.
- **Messagerie instantanée** : possibilité d'envoyer et de recevoir des messages en temps réel entre utilisateurs (discussions individuelles ou en groupe).
- **Partage d'images** : échange simple et rapide de fichiers visuels (cours, annonces, capture de documents, problèmes à résoudre, etc.).
- **Appels vocaux** : fonctionnalité permettant d'organiser des réunions en ligne, d'abord en individuel puis en groupe, afin de faciliter la collaboration entre étudiants.

Fonction Opérationnelle :

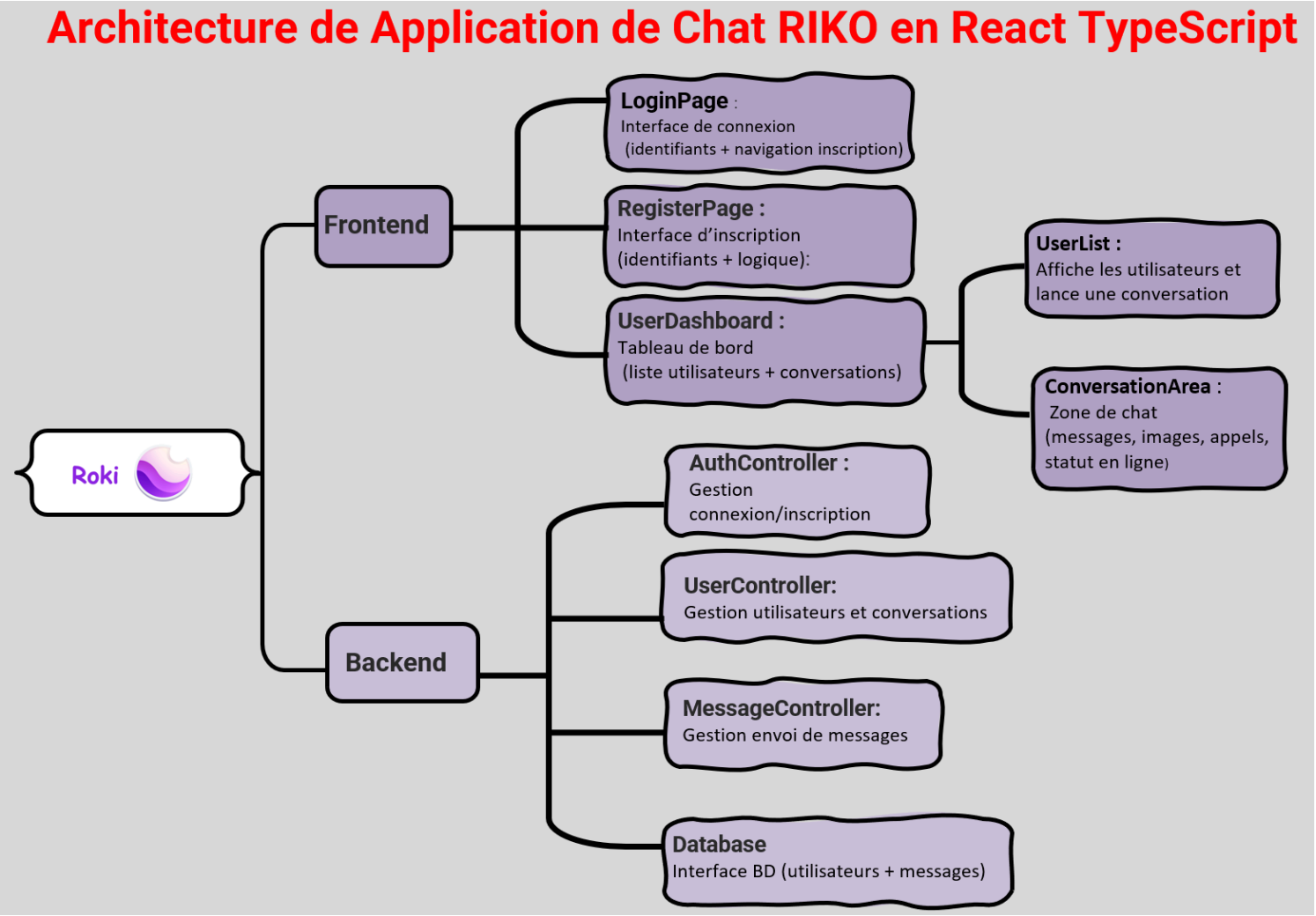
- Notifications
- barre de recherche utilisateur
- web socket
- affiche si la personne est en ligne

1.4 Contraintes identifiées

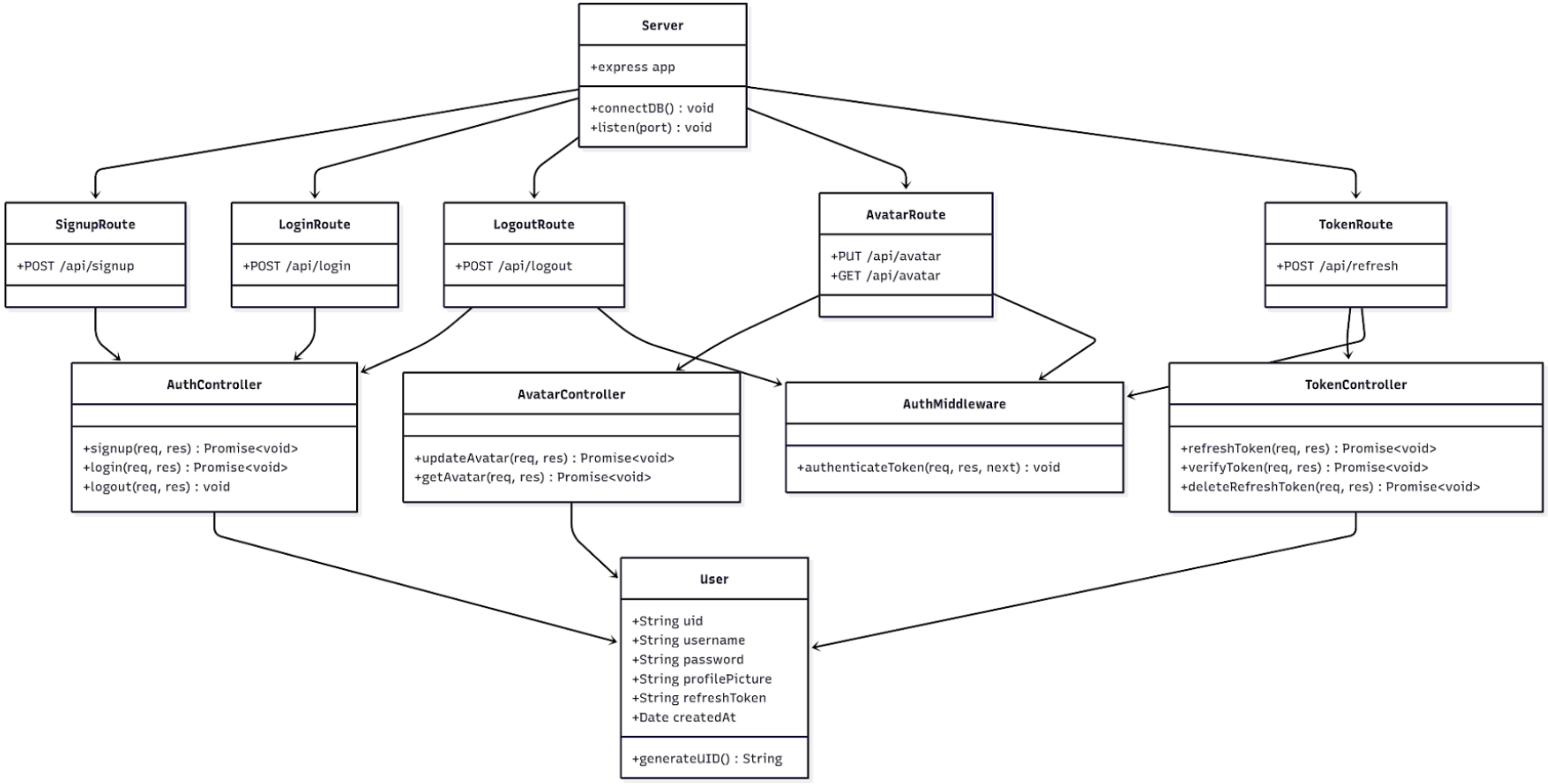
Aucune contrainte majeure n'a été constatée au cours du projet. Toutefois, certaines difficultés sont apparues dans la partie **Frontend**.

En effet, l'apprentissage du **framework ReactJS** a nécessité un temps d'adaptation pour bien comprendre sa logique et ses bonnes pratiques. De plus, la conception du **dashboard**, notamment l'affichage d'une liste d'utilisateurs, a représenté un défi supplémentaire. Enfin, la **configuration et l'intégration de la zone de chat** ont également demandé des efforts particuliers afin d'assurer son bon fonctionnement et son intégration avec le reste du système.

1.5 Schémas de L'architecture du Projet



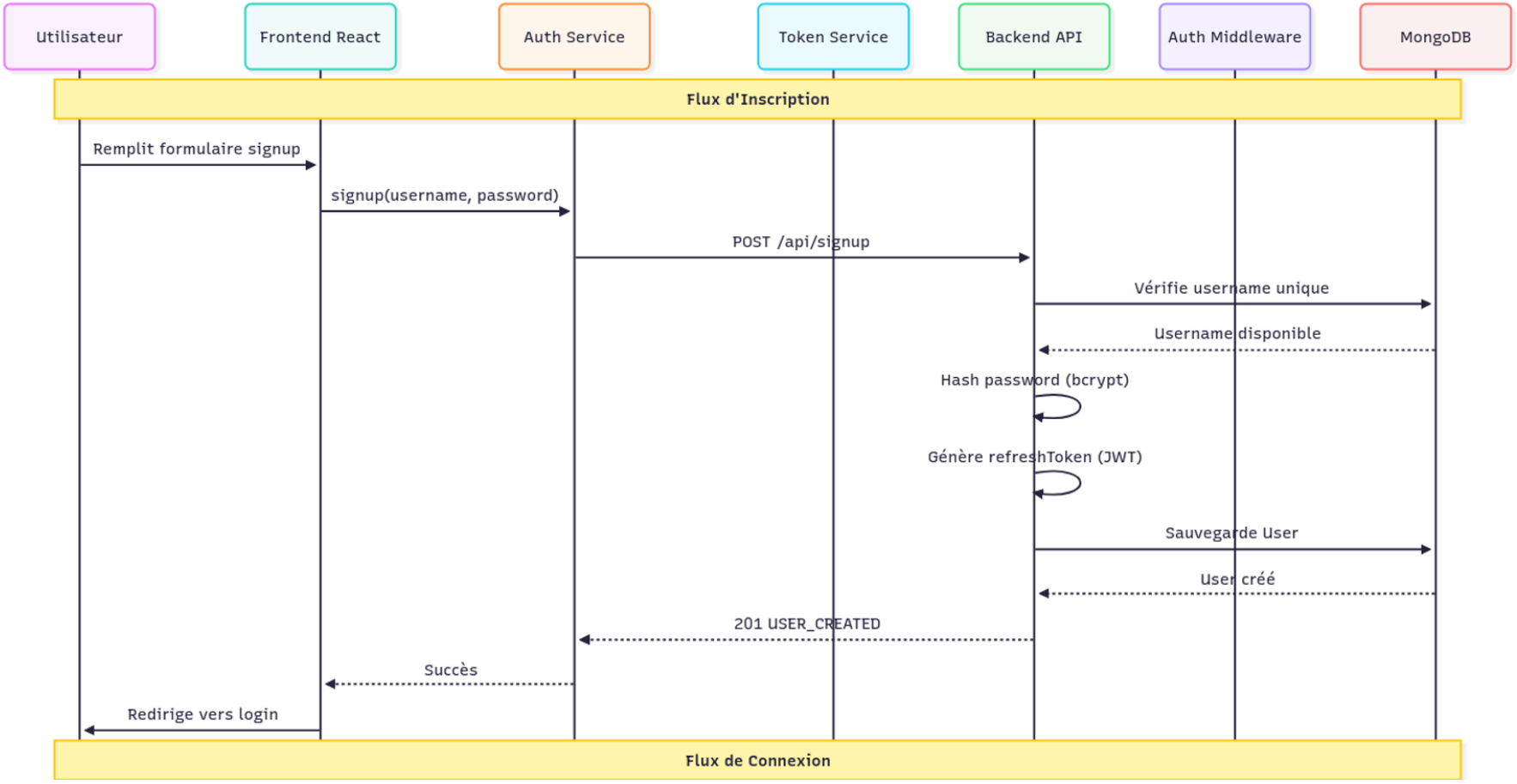
1.6 Diagrammes de Classes Backend

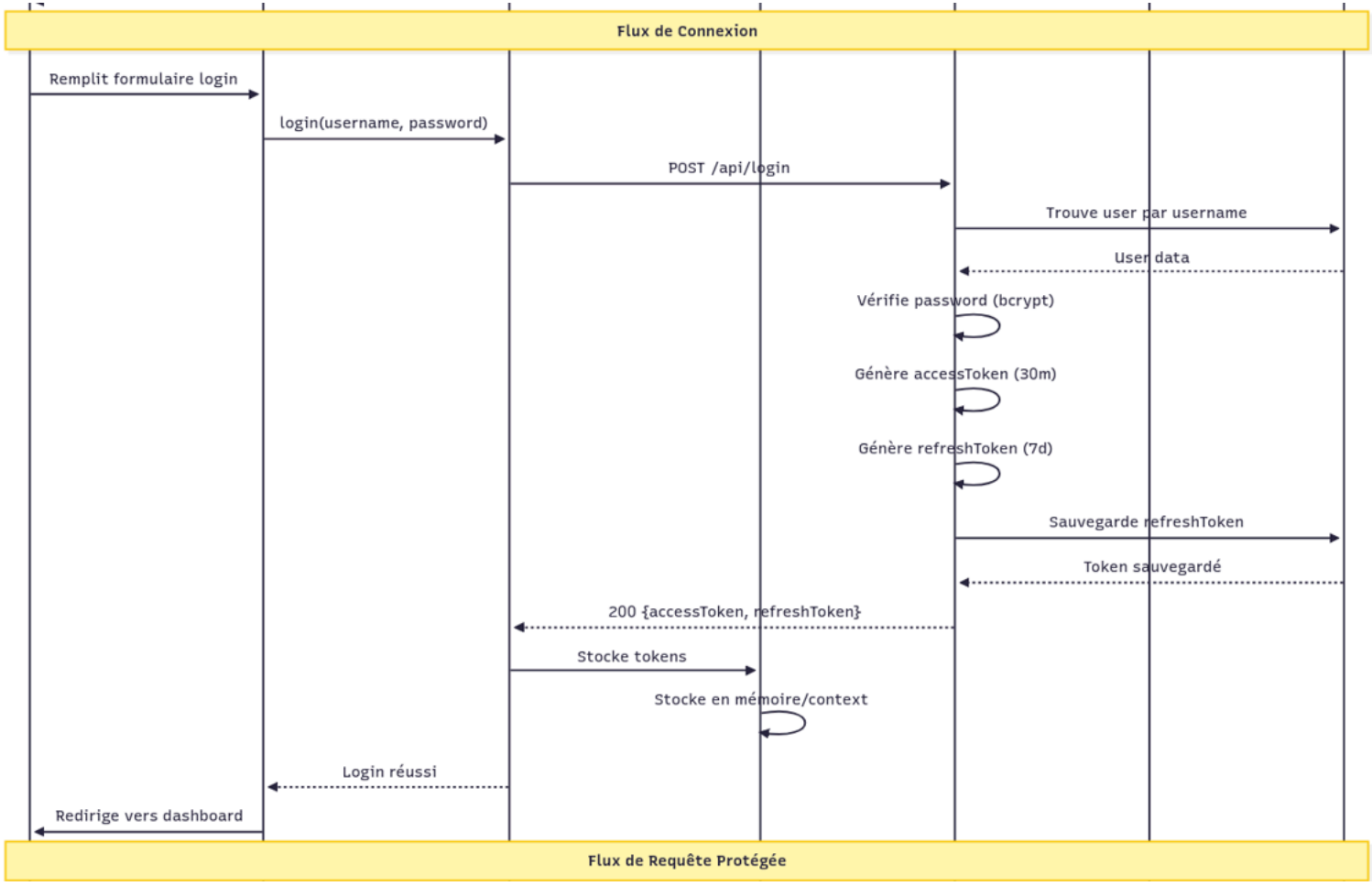


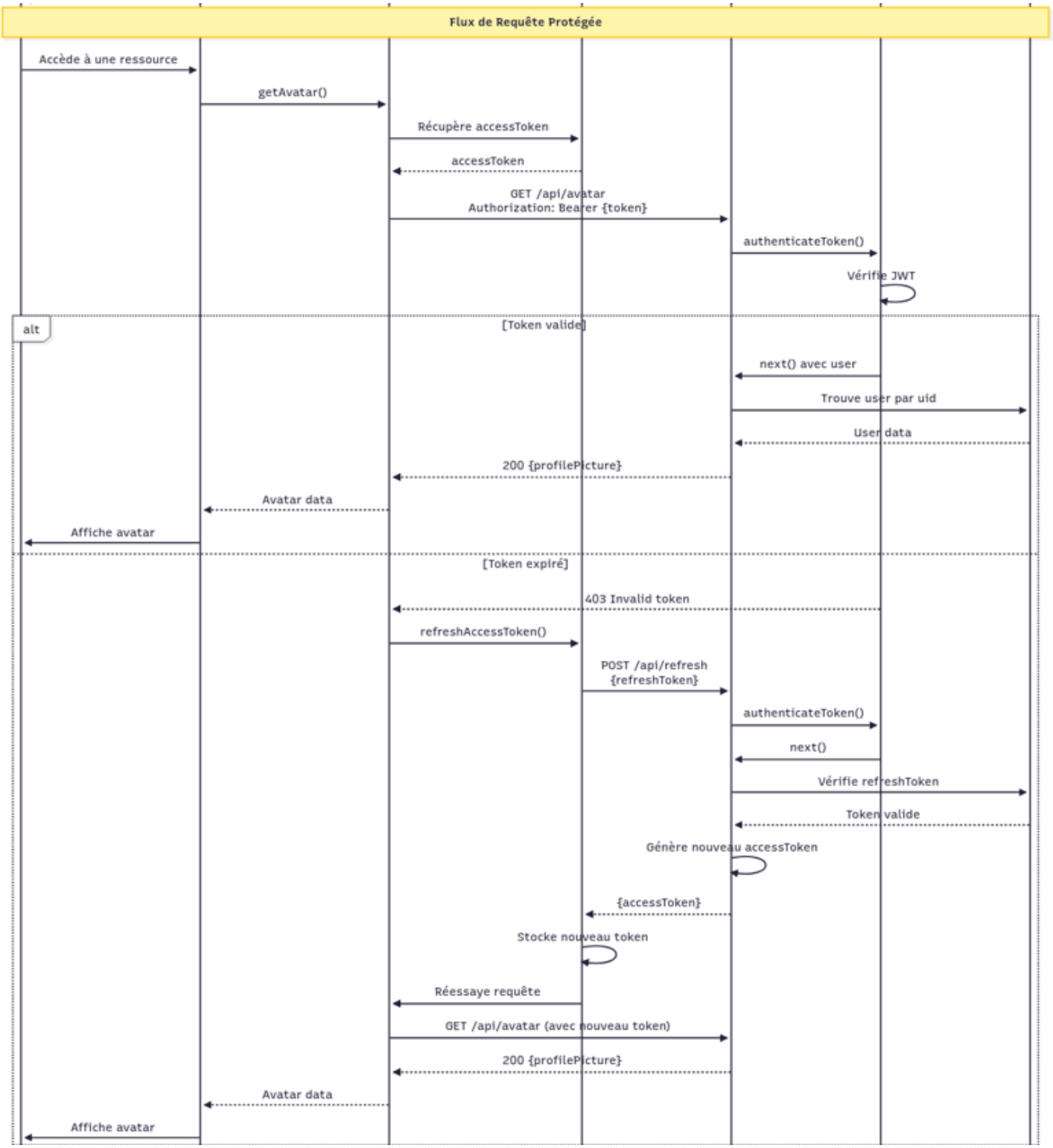
1.7 Cas d'utilisation

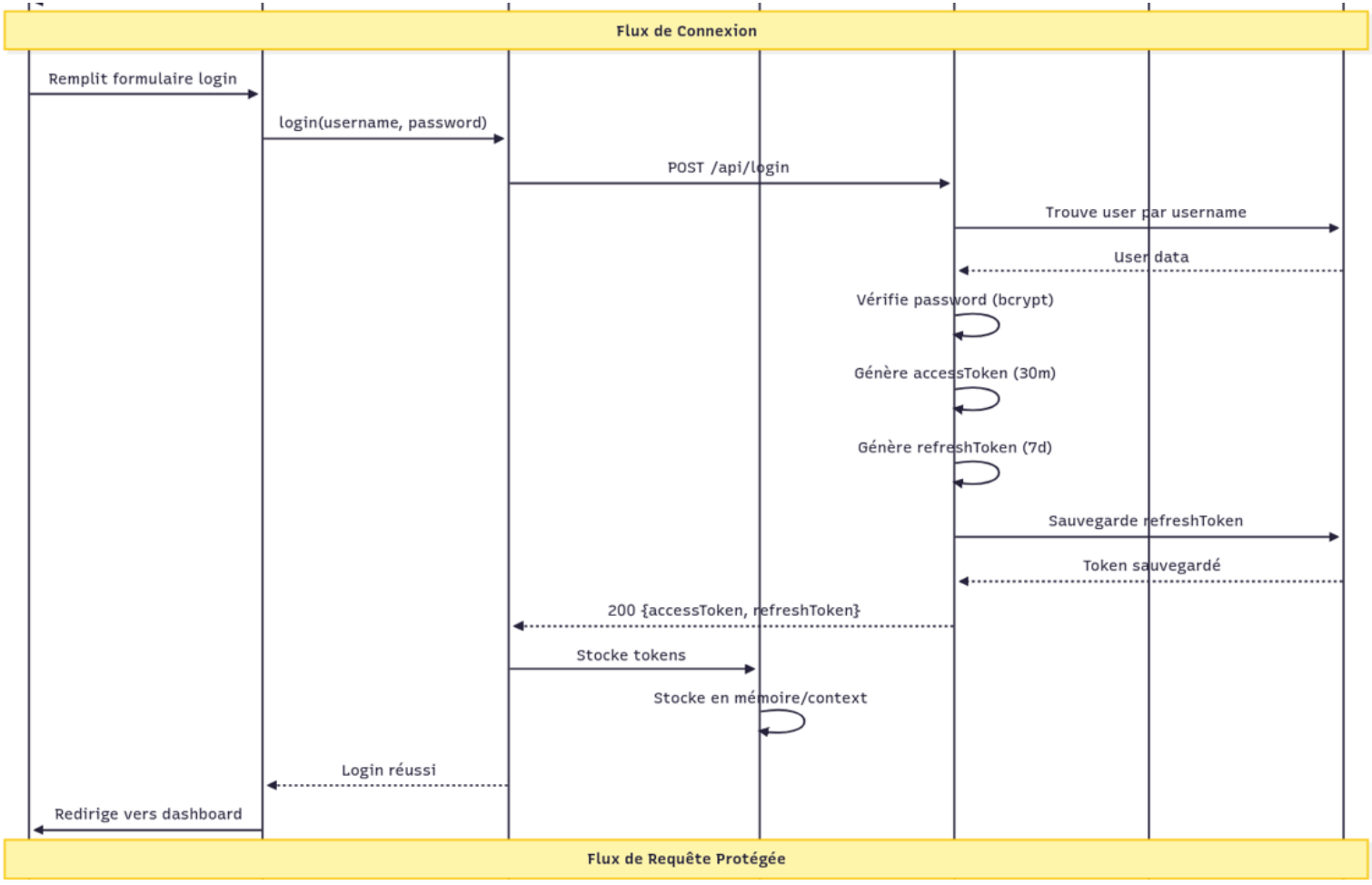
Imaginez deux étudiants en Réseaux & Télécommunications en plein TP de groupe. Ils rencontrent une difficulté technique et, au lieu de perdre du temps entre mails, SMS ou autres supports, ils ouvrent **Moki**. En quelques clics, ils se connectent, partagent instantanément leur fiche TP en photo, décrivent le problème et collaborent directement sur la plateforme. Résultat : une discussion claire, des échanges rapides, la possibilité d'ajouter des images, et même un appel vocal si besoin. Avec **Moki**, les étudiants disposent d'un espace unique qui simplifie la communication, centralise les documents et accélère la résolution des problèmes. Un outil pensé pour l'efficacité et le travail collaboratif.

1.8 Diagrammes de séquence




Flux de Connexion
Remplit formulaire login
login(username, password)
POST /api/login
Trouve user par username
User data
Vérifie password (bcrypt)
Génère accessToken (30m)
Génère refreshToken (7d)
Sauvegarde refreshToken
Token sauvegardé
200 {accessToken, refreshToken}
Stocke tokens
Stocke en mémoire/context
Login réussi
Redirige vers dashboard
Flux de Requête Protégée


Flux de Requête Protégée
Accède à une ressource
getAvatar()
Récupère accessToken
accessToken
GET /api/avatar
Authorization: Bearer {token}
authenticateToken()
Vérifie JWT
alt
[Token valide]
next() avec user
Trouve user par uid
User data
200 {profilePicture}
Avatar data
Affiche avatar
[Token expiré]
403 Invalid token
refreshAccessToken()
POST /api/refresh
{refreshToken}
authenticateToken()
next()
Vérifie refreshToken
Token valide
Génère nouveau accessToken
{accessToken}
Stocke nouveau token
Réessaye requête
GET /api/avatar (avec nouveau token)
200 {profilePicture}
Avatar data
Affiche avatar


Flux de Connexion
Remplit formulaire login
login(username, password)
POST /api/login
Trouve user par username
User data
Vérifie password (bcrypt)
Génère accessToken (30m)
Génère refreshToken (7d)
Sauvegarde refreshToken
Token sauvegardé
200 {accessToken, refreshToken}
Stocke tokens
Stocke en mémoire/context
Login réussi
Redirige vers dashboard
Flux de Requête Protégée

1.9 Choix technologiques

Nous avons choisi **Node.js** et **React avec TypeScript (TSX)** pour développer une application moderne, dynamique et maintenable.

React permet d'avoir une interface réactive qui s'adapte facilement aux changements de valeurs, tandis que TypeScript renforce la fiabilité du code grâce au typage statique. Cette combinaison offre une structure de projet claire, basée sur des composants réutilisables et modulaires, ce qui rend le développement plus flexible et évolutif.

1.10 Justification des langages

Nous avons choisi **TypeScript** et **React** car ils offrent de nombreux avantages pour le développement de notre application.

- ➔ **Réduction des bugs** : TypeScript détecte les erreurs de typage à la compilation, ce qui limite les bugs avant l'exécution.
- ➔ **Meilleure maintenabilité** : Le typage statique rend le code plus clair, structuré et facile à modifier.
- ➔ **Gain de productivité** : L'autocomplétion et la documentation intégrée facilitent le travail des développeurs.
- ➔ **Typage des Hooks React** : Permet de mieux contrôler les états et effets, rendant le code plus fiable.
- ➔ **Temps de développement plus long** : Le typage strict demande plus de préparation au départ.
- ➔ **Configuration complexe** : La mise en place de TypeScript requiert des fichiers et dépendances supplémentaires.
- ➔ **Erreurs parfois verbeuses** : Certaines erreurs de typage peuvent être trop détaillées et difficiles à interpréter.

1.11 Frameworks

Pour le front-end, on utilise **React avec TypeScript**, ce qui nous permet de créer des interfaces dynamiques et modulaires très facilement.

Pour le back-end, on utilise **Node.js avec Express**. Express nous aide à gérer les routes et les requêtes rapidement et simplement, comme pour l'inscription, la connexion et la gestion des avatars. C'est le cœur de notre serveur et ça facilite beaucoup le développement

1.12 Persistance des données (JSON)

Pour la persistance des données, nous utilisons **MongoDB**, une base de données **NoSQL** orientée documents.
Elle permet de stocker des données sous forme de **JSON**, ce qui offre une grande flexibilité et facilite l'évolution du schéma de l'application.
Cette solution est particulièrement adaptée aux projets avec des données **dynamiques et non strictement structurées**, tout en garantissant des performances élevées pour les lectures et écritures.*

2. l'organisation du travail

2.1 répartition des tâches

Tâche	Descriptif	Réalisée par
Tâche O1: Conception du projet		
T1.1	Élaboration du cahiers de charge (Besoin,fonctions,cas d'utilisation…)	IMANE
T1.2	Réalisation du schéma de l'architecture du projet.	IMANE
T1.3	Réalisation du diagrammes de Classes Backend	Soihiri
T1.4	Réalisation du Diagrammes de séquence	Soihiri
T1.4	Etude et choix de la technologie du Projet	Soihiri
T1.5	Etude et choix d'une bases de donnée du Projet	Thomas
T1.6	Définitions de la charte graphique (couleurs, typographie, icônes)	IMANE

T1.7	Etude et conception page de l'applications	Thomas
T1.8	Réalisations du Rapport de conceptions	Thomas
Tâche O2: Développement du projet		
T2.1	Réalisation de la fiche et suivi de tâches	Imane
T2.2	Mise en place du projet React (structure, composants, routes)	Thomas
T2.3	Création des interfaces principales (connexion, inscription, profil, dashboard)	Thomas
T2.4	Étude de la cohérence entre la charte graphique et les maquettes proposées	Imane
T2.5	Mise en place du serveur Express et configuration de MongoDB	Soihiri
T2.6	Création des routes backend (authentification, gestion des utilisateurs, avatar)	Soihiri
T2.7	Liaison Front-End / Back-End via les requêtes API	Soihiri /Thomas
T2.8	Réalisations du Rapport du rapport d'avancement du Projet	Imane

Tâche O3: Tests, débogage et finalisation		
T3.1	Écriture et exécution des tests unitaires frontend (composants, hooks).	Thomas
T3.2	Écriture et exécution des tests unitaires et d'intégration backend (routes, contrôleurs, DB).	Soihiri
T3.3	Tests end-to-end (inscription, connexion, upload/avatar, refresh token) et validation fonctionnelle.	Imane/ Thomas / Soihiri
T3.4	Correction des bugs front-end remontés pendant les tests (UI, formulaires, responsive).	Thomas
T3.5	Correction des bugs back-end (API, authentification, gestion des tokens, base MongoDB).	Soihiri
T3.6	Réalisations du Rapport Finale du Projet	Imane

2.2 Tableau de suivi du projet

Le suivi du projet a été réalisé principalement via **emails** et **Google Docs**, avec un tableau collaboratif présenté ci-dessous.

Tâche	Réalisations	Commentaire
Tâche O1: Conception du projet		
T1.1	Terminé	
T1.2	Terminé	
T1.3	Terminé	
T1.4	Terminé	
T1.4	Terminé	
T1.5	Terminé	
T1.6	Terminé	
T1.7	Terminé	
T1.8	Terminé	**Compte rendu déposé le sam. 4 oct. à 17h47**
Tâche O2: Développement du projet		
T2.1	Terminé	
T2.2	Terminé	**dossiers en place : components,hooks,pages,assets,types et contexts**
T2.3	Terminé	
T2.4	En cours	
T2.5	Terminé	**utilisation MongoDB_Atlas en ligne**
T2.6	Terminé	**Route déjà en place attente mises à jour front-end pour vérifications**

T2.7	En cours	
T2.8	En cours	
Tâche O3: Tests, débogage et finalisation		
T3.1	En cours	
T3.2	Terminé	
T3.3	Pas commencé	
T3.4	Pas commencé	
T3.5	En cours	
T3.6	En cours	

3. l'état d'avancement technique du serveur et du client

3.1 architecture mise en place

Notre projet est une application web moderne full-stack, combinant un Front-End réactif en React et un Back-End robuste en Node.js / Express, connecté à une base de données MongoDB Atlas. L'architecture suit le modèle client-serveur classique et utilise des API REST sécurisées par JWT, garantissant à la fois performance et sécurité.

3.1.1 Etat d'avancement du Serveur

Le serveur est déjà très fonctionnel et permet :

- **Inscription et connexion sécurisées** des utilisateurs avec génération de tokens JWT.
- **Gestion des avatars** : upload et récupération rapide des images de profil.
- **Renouvellement automatique des tokens** pour une expérience utilisateur fluide.
- Routes protégées accessibles uniquement avec un token valide.

Les middlewares pour le CORS, le parsing des requêtes et l'authentification sont en place. La connexion à la base de données MongoDB Atlas fonctionne parfaitement, et les tests unitaires backend sont terminés. Le serveur est prêt à supporter toutes les fonctionnalités de l'application.

3.1.2 Etat d'avancement du Client

Le client est développé avec React.js et TypeScript, en exploitant des hooks personnalisés et le contexte API pour gérer l'état global.
La structure est claire et organisée : **components, pages**, **hooks**, **contexts**, **assets** et **types**. Les pages principales déjà implémentées permettent une navigation fluide et intuitive

- **Page d'accueil** avec un message d'accroche et un bouton « Commencer ».
- **Page de connexion** simple et rapide, avec lien vers l'inscription.
- **Page d'inscription** pour créer facilement un compte.
- **Tableau de bord** regroupant conversations, messages et statut utilisateur.

L'intégration avec le backend est en cours, et la cohérence avec la charte graphique est en train d'être vérifiée pour offrir une interface élégante et uniforme.

3.1.3 Etat d'avancement Global

Le projet **est déjà entièrement fonctionnel pour les opérations de base** : inscription, connexion, gestion des tokens et affichage du dashboard.
Le backend est sécurisé et fiable, le frontend est intuitif et agréable à utiliser, et l'intégration complète **Front-End / Back-End** progresse rapidement. Manque plus que les tests unitaires backend, et les tests frontend ainsi que les tests end-to-end pour assurer une expérience utilisateur optimale.

Des fonctionnalités avancées comme upload d'avatar, refresh token automatique et chat en temps réel sont déjà en développement et seront bientôt pleinement opérationnelles.

3.2 fonctionnalités déjà implémentés

Backend :

- Inscription et authentification sécurisée.
- Gestion complète des tokens JWT (access + refresh).
- Routes protégées et middleware d'authentification.
- Upload et récupération des avatars.
- Gestion CRUD des utilisateurs.

Frontend :

- Pages d'accueil, connexion et inscription fluides et interactives.
- Tableau de bord dynamique avec liste de conversations et messages.
- Barre de recherche et zone de discussion en temps réel.
- Statut utilisateur visible et mis à jour instantanément.

Intégration Front-End / Back-End :

- Formulaires de connexion et d'inscription connectés au serveur.
- Gestion des tokens pour sécuriser l'accès aux pages protégées.
- Mise à jour en temps réel du tableau de bord après connexion.

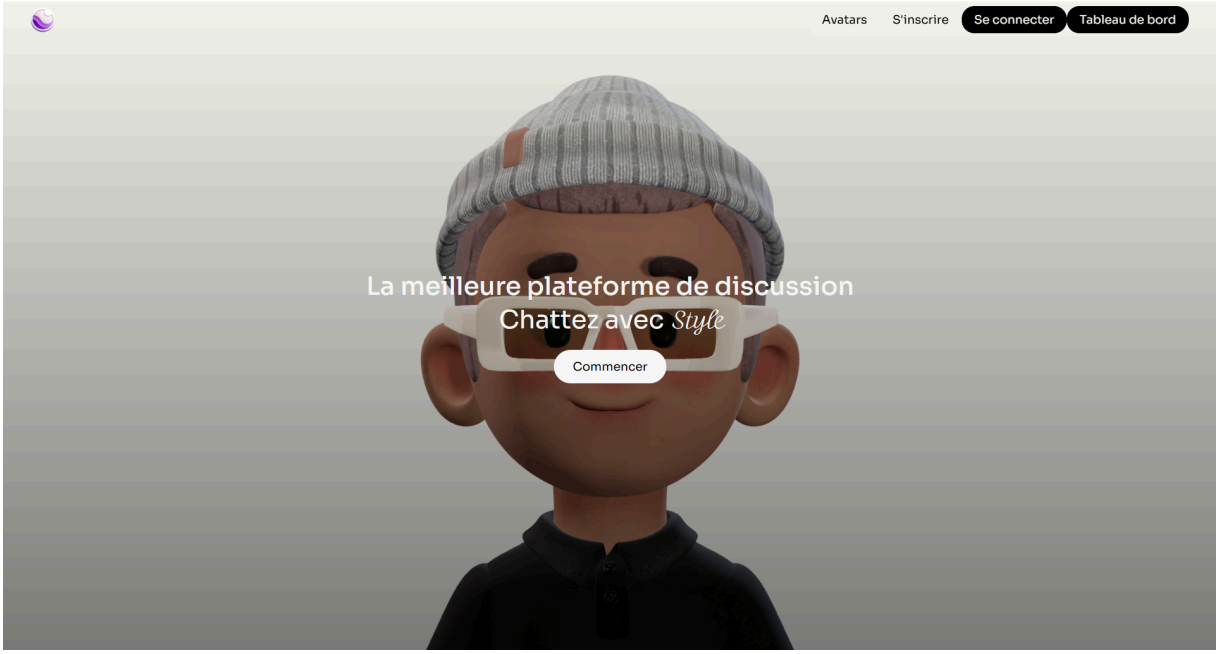
3.3 captures d'écran

2.3.1 Page d'accueil

Dès votre arrivée sur notre site, vous êtes automatiquement redirigé vers la page d'accueil. Celle-ci présente brièvement notre application avec un message accrocheur : **« La meilleure plateforme de discussion — Chattez avec style »**.

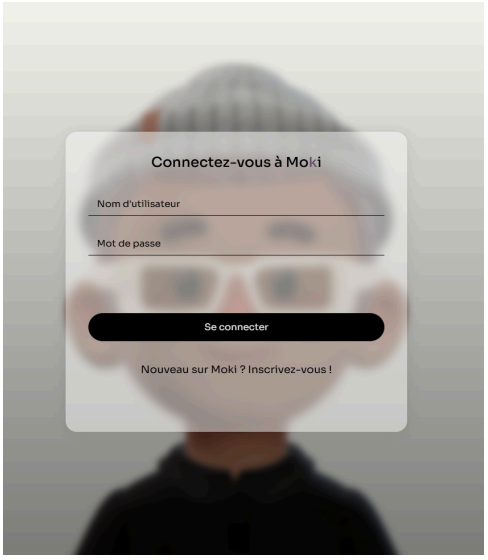
Juste en dessous, un bouton **« Commencer »** vous invite à accéder à la page de connexion, où vous trouverez également une option pour **créer un compte** si vous n'êtes pas encore inscrit.

Enfin, dans le coin inférieur droit de la page, plusieurs boutons sont disponibles : **Avatar**, **S'inscrire**, **Se connecter** et **Tableau de bord** (ce dernier n'est accessible que lorsque vous êtes connecté)



3.3.2 Page de connexion

La page de connexion permet à l'utilisateur d'accéder à son espace personnel sur la plateforme **Moki**.
Elle se compose d'un formulaire simple et intuitif contenant les champs suivants :



Nom d'utilisateur : pour saisir votre identifiant personnel

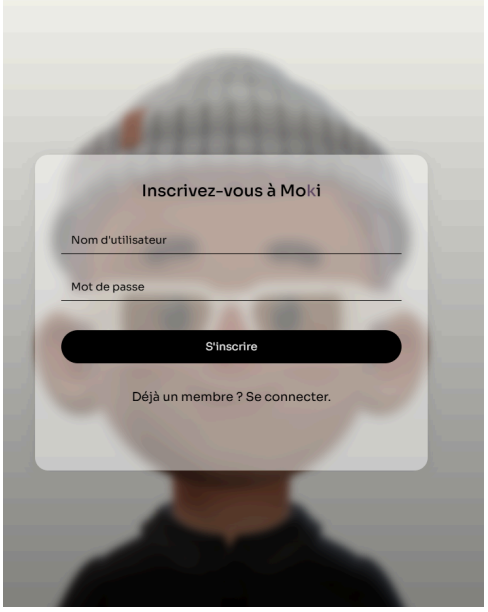
Mot de passe : pour entrer votre mot de passe de connexion

Une fois ces informations renseignées, il suffit de cliquer sur le bouton « Se connecter » pour accéder à votre compte

En dessous du formulaire, un lien **« Nouveau sur Moki ? Inscrivez-vous ! »**
permet aux nouveaux utilisateurs de créer facilement un compte s'ils n'en possèdent pas encore.

3.3.3 Page d'inscription

La page d'inscription permet aux nouveaux utilisateurs de créer un compte sur la plateforme **Moki**.
Elle se présente sous la forme d'un formulaire comprenant les champs suivants :



Nom d'utilisateur : pour définir votre identifiant personnel.

Mot de passe : pour choisir votre mot de passe de connexion

Après avoir rempli ces informations, il suffit de cliquer sur le bouton « S'inscrire » pour valider la création du compte.

Enfin, un lien situé en bas de la page **« Déjà un membre ? Se connecter. »** permet de retourner facilement à la page de connexion pour les utilisateurs disposant déjà d'un compte.

3.3.4 Tableau de bord

Le tableau de bord est l'espace principal de l'application Moki, accessible après connexion. Il regroupe plusieurs sections permettant à l'utilisateur de gérer et suivre ses conversations.

Partie « Messages »

Cette section affiche la liste des discussions existantes avec le nom des utilisateurs et le statut de chaque conversation.

En haut de la liste se trouve une barre de recherche (« Recherche... ») permettant de rechercher un utilisateur spécifique.

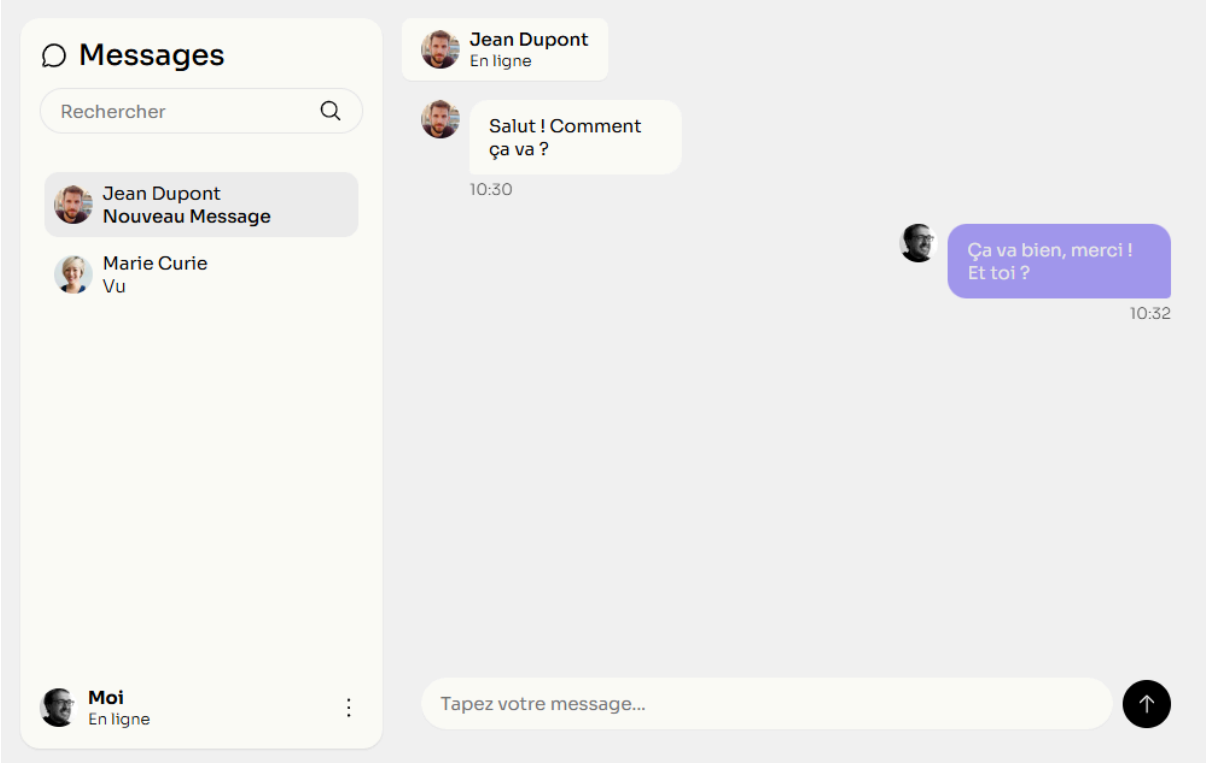
Partie « Conversation »

Cette zone affiche la bulle de discussion avec l'utilisateur sélectionné.

En bas, un champ de saisie permet d'écrire et d'envoyer un message.

Partie « Statut de l'utilisateur »

Dans cette partie, l'utilisateur connecté peut voir son propre état de connexion :



4. Aperçu du code

4.1 Modèle de données utilisateur (User Model)

Dans cette partie, nous avons défini le modèle de données de l'utilisateur à l'aide de la dépendance Mongoose.
Ce modèle contient plusieurs champs essentiels tels qu'un identifiant unique généré automatiquement, un nom d'utilisateur, un mot de passe, un avatar, un jeton de rafraîchissement et une date de création.
Nous avons également ajouté des fonctions pour générer l'identifiant avant l'enregistrement et pour gérer les erreurs en cas de doublon du nom d'utilisateur.

```
import mongoose from 'mongoose';
import { randomBytes } from 'crypto';


interface IUser {
    uid: string;
    username: string;
    password: string;
    profilePicture: string;
    refreshToken?: string;
    createdAt: Date;
}


const generateUID = (): string => {
    return 'u' + randomBytes(4).toString('hex');
}


const userSchema = new mongoose.Schema<IUser>({
    uid: { type: String, default: generateUID },
    username: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    profilePicture: { type: String, default: '' },
    refreshToken: { type: String, default: null },
    createdAt: { type: Date, default: Date.now }
});
```

```
userSchema.pre('save', function (next: any) {
    if (!this.uid) {
        this.uid = generateUID();
    }
    next();
});


userSchema.post('save', function (error: any, doc: any, next: any) {
    if (error.code === 11000) {
        next(new Error('Username already exists'));
    } else {
        next(error);
    }
});


export const User = mongoose.model(User', userSchema);
```

4.2 Point d'entrée du serveur backend (server.ts)

Ce code permet de configurer et lancer le serveur backend avec Express, de connecter la base de données MongoDB, d'ajouter les middlewares pour le CORS et le parsing des requêtes, et d'enregistrer les routes pour l'inscription, la connexion, l'avatar, le rafraîchissement de token, la déconnexion et les opérations utilisateur.

Point d'entrée du serveur backend : server.ts

```
import dotenv from 'dotenv';
dotenv.config();

import express from 'express';
import cors from 'cors';
import bodyParser from 'body-parser';
import signupRoute from './routes/signupRoute';
import loginRoute from './routes/loginRoute';
import avatarRoute from './routes/avatarRoute';
import tokenRoute from './routes/tokenRoute';
import logoutRoute from './routes/logoutRoute';
import userRoute from './routes/userRoute';
```

```
import connectDB from './config/db';
import { corsOptions, corsMiddleware } from './config/cors';

const app = express();

connectDB();

app.use(cors(corsOptions));
app.use(corsMiddleware);
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));


app.use('/api/signup', signupRoute);
app.use('/api/login', loginRoute);
app.use('/api/avatar', avatarRoute);
app.use('/api/refresh', tokenRoute);
app.use('/api/logout', logoutRoute);
app.use('/api/protect', userRoute);


app.listen(3000, () => {
    console.log('Server is running on port 3000');
});
```

4.4 Connexion à la base de données (db.ts)

Ce code permet de connecter le serveur à la base de données MongoDB, d'utiliser l'URI définie dans les variables d'environnement ou une valeur par défaut, et d'afficher un message pour indiquer si la connexion a réussi ou échoue.

Connexion à la base de données : models/userModel.ts

```
import mongoose from 'mongoose';

const dbURI = process.env.DB_URI || "mongodb://localhost:27017/moki";
```

```
const connectDB = async () => {
    try {
        await mongoose.connect(dbURI);
        console.log(" Connected to MongoDB");
    } catch (error) {
        console.error(" Error connecting to MongoDB:", error);
    }
}

export default connectDB;
```

4.5 Contrôleur de présence du token

Ce code permet de vérifier la présence et la validité du token JWT dans les requêtes HTTP.
Il récupère le token depuis l'en-tête Authorization, renvoie une erreur si le token est absent, et utilise jwt.verify pour vérifier sa validité.
Si le token est valide, il ajoute les informations de l'utilisateur à la requête (req.user) et passe au middleware suivant, permettant ainsi de protéger les routes de l'application.

```
Contrôleur de présence du token : midleware/auth.ts
```

```
import jwt from 'jsonwebtoken';

export const authenticateToken = (req: any, res: any, next: any) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {
        return res.status(401).json({ error: 'Access token required' });
    }

    jwt.verify(token, process.env.SECRET_KEY!, (err: any, user: any) =>
{
        if (err) {
            return res.status(403).json({ error: 'Invalid token' });
        }
```

```
        req.user = user;
        next();
    });
};
```

4.6 Exemple de route protégée

Ce code permet de définir une route protégée sur le serveur backend.
Il utilise Express Router et applique le middleware authenticateToken pour vérifier la présence et la validité du token JWT avant d'exécuter la fonction protect du contrôleur.
Ainsi, seules les requêtes avec un token valide peuvent accéder aux opérations définies sur cette route.

Exemples de route : routes/userRoute.ts

```
import express from 'express';
import { protect } from '../controllers/authController';
import {authenticateToken} from '../middleware/auth';

const router = express.Router();

router.post('/', authenticateToken, protect);

export default router;
```

4.7 Mise en place du CORS

Ce code permet de configurer le CORS pour le serveur backend afin de contrôler l'accès aux ressources depuis des origines différentes.
Nous avons défini corsOptions pour autoriser certaines origines, méthodes et en-têtes, et créé un middleware corsMiddleware qui ajoute les headers nécessaires à chaque requête pour gérer les autorisations cross-origin.

Mise en place du cors : config/cors.ts

```
import { CorsOptions } from 'cors';

export const corsOptions: CorsOptions = {
    origin: ['http://localhost:3000', 'http://localhost:3001'],
```

```
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization']
};

export const corsMiddleware = (req: any, res: any, next: any) => {
    res.header('Access-Control-Allow-Origin', '*');
    res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE,
OPTIONS');
    res.header('Access-Control-Allow-Headers', 'Content-Type,
Authorization');
    next();
};
```

4.8 Vérification du token utilisateur

Ce code permet de vérifier la validité d'un token JWT envoyé par l'utilisateur.
Il récupère le token depuis le corps de la requête, renvoie une erreur si le token est absent, puis utilise jwt.verify pour décoder le token et retrouver l'utilisateur correspondant dans la base de données.

```
Connexion utilisateur : controllers/tokenController.ts
```

```
export const verifyToken = async (req: Request, res: Response) => {
    const { token } = req.body;

    if (!token) {
        res.status(401).json({ message: 'Token is required', code:
"TOKEN_MISSING" });
        return;
    }
    try {
        const decoded = jwt.verify(token, process.env.SECRET || "dev")
as { id: string };
        const user = await User.findById(decoded.id);
        if (!user) {
```

```
            res.status(401).json({ message: 'User not found', code:
"USER_NOT_FOUND" });
            return;
        }
        res.status(200).json({ message: 'Token is valid', code:
"TOKEN_VALID" });
        return;
    } catch (error) {
        res.status(401).json({ message: 'Invalid token', code:
"INVALID_TOKEN" });
        return;
    }
};
```

Le résultat renvoie une réponse indiquant si le token est valide, invalide ou si l'utilisateur n'existe pas, permettant ainsi de sécuriser l'accès aux fonctionnalités protégées.

4.9 Authentification de l'utilisateur

Ce code permet d' authentifier un utilisateur lors de la connexion.
Il recherche l'utilisateur dans la base de données grâce au nom d'utilisateur, vérifie la validité du mot de passe avec bcrypt.compare, puis génère un accès token et un refresh token avec JWT.
Le refresh token est enregistré dans la base de données pour permettre le renouvellement du token d'accès, et une réponse JSON est renvoyée pour indiquer la réussite de la connexion et fournir les tokens à l'utilisateur.

```
Exemple du code de l'authentification de l'user
:controllers/authController.ts
```

```
const user = await User.findOne({ username });
    if (!user) {
        res.status(401).json({ error: 'Invalid credentials', code:
"INVALID_CREDENTIALS" });
        return;
    }

const isValidPassword = await bcrypt.compare(password, user.password);
```

```
    if (!isValidPassword) {
        res.status(401).json({ error: 'Invalid credentials', code:
"INVALID_CREDENTIALS" });
        return;
    }

const accessToken = jwt.sign(
    { uid: user.uid, username: user.username },
            secretKey,
            { expiresIn: '30m' }
        );
        const refreshToken = jwt.sign(
            { uid: user.uid, username: user.username },
            refreshSecret,
            { expiresIn: '7d' }
        );

        user.refreshToken = refreshToken;
        await user.save();

        const token = {
            accessToken,
            refreshToken
        };

        res.json({
            message: 'Login successful',
            token,
            code: "LOGIN_SUCCESS"
        });
```

4.10 Dashboard utilisateur (Dashboard.tsx)

Ce code permet de gérer l'affichage du tableau de bord de l'utilisateur dans l'application React.
Il initialise un état de conversations contenant des informations sur les conversations et leurs messages, utilise le contexte d'authentification pour récupérer l'utilisateur et les tokens, et redirige vers la page de connexion si les tokens sont absents.

Le hook useEffect permet également d'exécuter la fonction protect pour sécuriser l'accès au tableau de bord et garantir que seules les personnes authentifiées peuvent y accéder.

```
import React, { useEffect, useState } from "react";
import "../styles/dashboard.css";
import { useAuthContext } from "contexts/AuthContext";
import useAuth from "../hooks/useAuth"
import { useNavigate } from "react-router-dom";
import Avatars from "../components/Avatars";


interface Message {
  id: number;
  sender: "me" | "other";
  text: string;
}


interface Conversation {
  id: number;
  name: string;
  status: string;
  messages: Message[];
}


const Dashboard: React.FC = () => {
  const [conversations, setConversations] = useState<Conversation[]>([
    {
      id: 1,
      name: "Ruben Merrick",
      status: "Nouveau message",
      messages: [],
```

```
    },
    {
      id: 2,
      name: "Adeline Griffis",
      status: "Nouveau message",
      messages: [],
    },
    {
      id: 3,
      name: "Lyda Townsend",
      status: "Vu",
      messages: [
        { id: 1, sender: "other" as const, text: "Salut !" },
        { id: 2, sender: "me" as const, text: "Coucou 😊" },
      ],
    },
  ]);
  const { keys, user } = useAuthContext();
  const navigate = useNavigate();
  const { protect } = useAuth();


   useEffect(()=> {
    if(!keys.accessToken || !keys.refreshToken) {
      navigate('/login');
    }

    protect();


    // if (!user?.profileImage) {
    //   navigate('/avatar')
    // }


  }, []);
```

4.11 Page de connexion utilisateur (Login.tsx)

Ce code permet de gérer l'interface et la logique de connexion de l'utilisateur dans l'application React.

Il récupère les identifiants saisis (nom d'utilisateur et mot de passe), utilise le hook useAuth pour appeler la fonction login, et redirige vers le tableau de bord (/dashboard) si la connexion est réussie.

Le composant surveille également l'état des tokens dans le contexte d'authentification pour empêcher l'accès à la page de connexion si l'utilisateur est déjà authentifié, et affiche les messages d'erreur éventuels pour guider l'utilisateur.

```
import React, { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import "../styles/login.css";
import useAuth from "hooks/useAuth";
import { useAuthContext } from "contexts/AuthContext";


const Login = () => {
    const { message, login } = useAuth();
    const [username, setUsername] = useState<string>("");
    const [password, setPassword] = useState<string>("");
    const {keys} = useAuthContext();
    const navigate = useNavigate();


    const handleLogin = async() => {
        console.log('Username : ', username);
        console.log('Password : ', password);

        const connect = await login(username, password);

        if (connect == "LOGIN_SUCCESS"){
        console.log("keys");

      navigate ("/dashboard");
    }
    }


  useEffect(()=>{
    if (keys.accessToken){
        console.log("keys");

      navigate ("/dashboard");
```

```
    }

  }, [keys])


  return (
    <>
    <main>
        <div className="container container-login">
            <div className="form-container">
                <section className="bold login-title">Connectez-vous à <Link to="/"
className="home">Mo<span className="k">k</span>i</Link></section>


                <form className="login-form">
                    <div className="form-group">


                        <input type="text" id="username" name="username" placeholder="Nom
d'utilisateur" value={username}
                        onChange={(event: React.ChangeEvent<HTMLInputElement>)=> {
                            setUsername(event.target.value);
                        }}
                        required />
                    </div>


                    <div className="form-group">

                        <input type="password" id="password" name="password"
value={password} placeholder="Mot de passe"
                            onChange={(event: React.ChangeEvent<HTMLInputElement>)=> {
                            setPassword(event.target.value);
                        }}
                        required />
                    </div>


                    <section className="message">{message}</section>


                    <button type="button" onClick={handleLogin}
className="button-primary">Se connecter</button>
                </form>
```

```
                <section className="main-description">
                    <p>Nouveau sur Moki ? <Link to="/Signup"
className="link-primary">Inscrivez-vous ! </Link></p>
                </section>
            </div>
        </div>
    </main>
    </>
  );
};


export default Login;
```